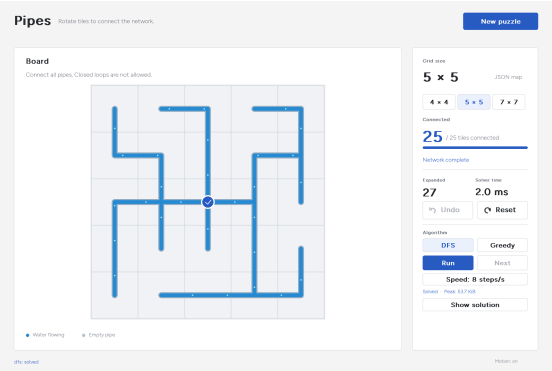
[TÊN TRƯỜNG]
[KHOA / BỘ MÔN]

BÀI TẬP LỚN 1 — TÌM KIẾM

Học kỳ 1, năm học 2026–2027

Môn: Nhập môn Trí tuệ nhân tạo

Pipes & Light Up:
Hiện thực giải thuật tìm kiếm cho hai bài toán Logic Puzzle



Danh sách thành viên

Họ và tên	MSSV	Lớp
[Họ tên 1]	[MSSV 1]	[Lớp 1]
[Họ tên 2]	[MSSV 2]	[Lớp 2]
[Họ tên 3]	[MSSV 3]	[Lớp 3]

Giảng viên hướng dẫn: [Tên giảng viên]

[Địa điểm], ngày 25 tháng 10 năm 2026

Mục lục

1 Mở đầu **2**

1.1 Mục tiêu . . . 2
1.2 Hai bài toán được chọn . . . 2
1.3 Môi trường và công cụ . . . 2
1.4 Tổ chức mã nguồn . . . 2
1.5 Quy trình tìm hiểu và hiện thực . . . 3

2 Bài toán 1: Pipes **3**

2.1 Tìm hiểu đề bài và luật chơi . . . 3
2.2 Phát biểu bài toán tìm kiếm . . . 4
2.3 Không gian tìm kiếm và cắt tỉa dùng chung . . . 4
2.4 Giải thuật tìm kiếm mù: Depth-First Search . . . 5
2.5 Giải thuật heuristic: Greedy Best-First . . . 5
2.6 Hiện thực và demo trực quan . . . 6
2.7 Tạo input . . . 6
2.8 Kết quả thực nghiệm: thời gian và bộ nhớ . . . 6
2.9 Hạn chế . . . 8

3 Bài toán 2: Light Up (Akari) **8**

3.1 Tìm hiểu đề bài và luật chơi . . . 8
3.2 Phát biểu bài toán tìm kiếm . . . 8
3.3 Giải thuật tìm kiếm mù: Depth-First Search . . . 9
3.4 Giải thuật heuristic: A* với cận dưới admissible . . . 9
3.5 Hiện thực và demo trực quan . . . 10
3.6 Tạo input . . . 10
3.7 Kết quả thực nghiệm: thời gian và bộ nhớ . . . 11
3.8 Hạn chế . . . 12

4 Kết luận **13**

1 Mở đầu

1.1 Mục tiêu

Bài tập lớn 1 yêu cầu nhóm chọn hai bài toán trong bộ Logic Puzzles tại https://www.puzzle-pipes.com/ (và các bài toán liên quan trên trang này), sau đó hiện thực bằng Python 3 hai giải thuật tìm kiếm cho **mỗi** bài toán: một giải thuật *tìm kiếm mù* (DFS hoặc BrFS) và một giải thuật *heuristic* (A*, Hill Climbing, v.v.). Bài làm phải có chế độ demo từng bước với hiển thị trực quan dạng game, kèm báo cáo trình bày quá trình tìm hiểu, quá trình hiện thực, bảng số liệu thời gian và bộ nhớ tiêu tốn của từng giải thuật trên từng input, và phần giải thích kết quả.

1.2 Hai bài toán được chọn

Nhóm chọn hai bài toán:

- **Pipes** (https://www.puzzle-pipes.com/): xoay các đoạn ống để tạo thành một mạng liên thông, không rò rỉ và không có chu trình. Hai giải thuật: tìm kiếm theo chiều sâu **DFS** (mù) và **Greedy Best-First** (heuristic).
- **Light Up / Akari** (https://www.puzzle-light-up.com/, cùng hệ bài toán Logic Puzzles): đặt bóng đèn trên ô trắng sao cho mọi ô trắng được chiếu sáng, không bóng nào chiếu thẳng vào bóng khác và mọi tường số có đúng số bóng kề. Hai giải thuật: **DFS** (mù) và **A*** (heuristic, cho lời giải tối ưu số bóng đèn).

Cả bốn giải thuật đều dùng chung mô hình phát biểu bài toán tìm kiếm trong không gian trạng thái và đều được đo trên cùng một bộ input với giới hạn tài nguyên giống nhau.

1.3 Môi trường và công cụ

- **Ngôn ngữ**: Python 3 (yêu cầu bắt buộc của đề bài). Môi trường phát triển dùng Python 3.13, quản lý phụ thuộc bằng `uv`.
- **Đồ họa và demo**: `pygame-ce` cho cả hai game; `pygame-gui` bổ sung điều khiển cho giao diện Light Up.
- **Kiểm thử và chất lượng mã**: `unittest` (50 test cho cả hai package), `ruff` kiểm tra phong cách mã; chạy tự động bằng GitHub Actions.
- **Đo lường**: `time.perf_counter` cho thời gian, `tracemalloc` cho đỉnh bộ nhớ cấp phát của Python.
- **Biên soạn tài liệu**: LaTeX (tectonic) cho báo cáo và slide.

1.4 Tổ chức mã nguồn

Mã nguồn đặt trong một kho chung, mỗi bài toán một package riêng biệt:

```
Introduction-to-AI/
  main.py             Launcher chung: uv run main.py [pipes|lightup]
  pipes_problem/      Game Pipes, 2 giai thuat, UI, benchmark, 33 test
```

```
light_up_problem/   Game Light Up, 2 giai thuat, UI, benchmark, 17 test
docs/               Phan tich hoc thuat, bang chung benchmark (CSV + md)
report/             Nguon LaTeX cua bao cao (main-vi, main-en)
slides/             Nguon LaTeX cua slides (main-vi, main-en)
```

Cách chạy chi tiết nằm trong `README.md` của kho. Hai game chạy bằng `uv run main.py pipes` và `uv run main.py lightup` (hoặc `python -m pipes_problem`, `python -m light_up_proble

1.5 Quy trình tìm hiểu và hiện thực

Nhóm thực hiện theo cùng một quy trình cho cả hai bài toán:

1. **Tìm hiểu luật chơi**: chơi trực tiếp trên trang gốc của từng puzzle, ghi lại luật bất biến (điều kiện thắng, ràng buộc), tham chiếu trang `puzzle-pipes.com` / `puzzle-light-up.com`.
2. **Phát biểu bài toán tìm kiếm**: xác định trạng thái, trạng thái đầu, tác động, chuyển tiếp, kiểm tra đích và chi phí; chọn cách biểu diễn trạng thái hiệu quả (tiền tố định hướng cho Pipes, bitmask cho Light Up).
3. **Thiết kế giải thuật**: một giải thuật mù (DFS) và một giải thuật heuristic (Greedy / A*), kèm bộ cắt tỉa hợp lệ dùng chung; viết giả mã trước khi code.
4. **Hiện thực và demo**: cài đặt game trực quan với pygame, có chế độ chạy từng bước (*step-by-step*) cho quá trình tìm kiếm và phát lại lời giải.
5. **Sinh input**: xây dựng bộ sinh puzzle theo seed xác định, kiểm chứng tồn tại nghiệm khi sinh; giải thích nguồn gốc từng input.
6. **Đo lường**: chạy benchmark tự động, ghi CSV thời gian – bộ nhớ – số node; lưu bằng chứng trong `docs/benchmarks/`.
7. **Phân tích**: giải thích số liệu, đối chiếu lý thuyết với thực nghiệm, ghi rõ hạn chế.

2 Bài toán 1: Pipes

2.1 Tìm hiểu đề bài và luật chơi

Nhóm tìm hiểu Pipes qua việc chơi trực tiếp trên https://www.puzzle-pipes.com/ và quan sát hành vi của game gốc. Sau quá trình chơi thử, nhóm tóm tắt luật như sau:

- Bàn chơi $m \times m$ gồm các ô ống. Mỗi ô có *hình dạng cố định* (ống thẳng, khuỷu, chữ T, chữ thập, đầu cụt) và một *định hướng* (hướng xoay) có thể thay đổi; xoay là tác động duy nhất của người chơi.
- Mỗi hình dạng có một tập cổng ở các phía Bắc–Đông–Nam–Tây. Xoay ô không đổi số cổng, chỉ đổi hướng.
- Mục tiêu: xoay sao cho **mọi** cổng mở đều gặp cổng mở tương ứng của ô kề (không cổng nào chìa ra ngoài biên, không cổng nào *dangling*), toàn bộ các ô nối thành **một** mạng liên thông và mạng **không chứa chu trình**.

- Nguồn nước (ô đánh dấu trên game gốc) chỉ để trực quan hóa: khi mạng hoàn chỉnh, nước chảy từ nguồn tới mọi ô. Đổi vị trí nguồn không làm đổi tập các bàn hợp lệ.

Nhìn theo phân loại của giáo trình, đây là bài toán *một agent, tất định, quan sát được toàn phần, rời rạc, tĩnh*. Đây là bài toán *thỏa mãn* (satisfaction): mọi bàn hợp lệ đều là lời giải, bài toán không yêu cầu tối ưu số lần xoay. Trò chơi gốc cũng không tính điểm theo số bước xoay.

2.2 Phát biểu bài toán tìm kiếm

Gọi $n = m^2$, đánh số các ô theo thứ tự hàng–cột (row-major). Ô i có hình dạng cố định và miền định hướng R_i gồm các xoay phân biệt của hình dạng đó. Nhóm phát biểu bài toán như sau:

Thành phần	Định nghĩa trong cài đặt
Input	Bàn chơi đã xáo trộn (hình dạng + định hướng lưu sẵn), kích thước, nguồn
Trạng thái	Tiền tố $s = (a_0, \ldots, a_{k-1})$ các định hướng đã gán
Trạng thái đầu	Tiền tố rỗng $()$, biết trước toàn bộ hình dạng
Tác động	Gán một định hướng hợp lệ biên cho ô k (ô kế tiếp theo row-major)
Chuyển tiếp	$s' = s + (a_k)$, chỉ nhận khi qua hết các kiểm tra khả thi
Kiểm tra đích	$k = n$ và bàn hoàn chỉnh thỏa mọi luật
Chi phí	1 mỗi phép gán; không giải thuật nào dùng chi phí để xếp hạng
Lời giải	Một bộ định hướng đầy đủ hợp lệ

Bảng 1: Phát biểu bài toán tìm kiếm cho Pipes.

Điểm cần nhấn: *một bước của giải thuật* là gán **tuyệt đối** một định hướng cho ô kế tiếp, khác với *một bước của người chơi* là xoay 90 độ. Một phép gán có thể tương ứng 0–3 lần xoay. Cả hai giải thuật dùng cùng mô hình gán, nên việc so sánh chúng là công bằng. Nhóm cũng tách bạch trạng thái tìm kiếm (tiền tố rỗng) với bàn xáo trộn lưu trong file input: bàn xáo trộn cung cấp miền R_i và thứ tự duyệt các xoay, còn tìm kiếm luôn bắt đầu từ tiền tố rỗng, không dựa vào đáp án của generator.

2.3 Không gian tìm kiếm và cắt tỉa dùng chung

Số cách gán đầy đủ (chưa xét biên) là $U = \prod_i |R_i| \le 4^n$: ô khuỷu/T/đầu cụt có 4 xoay phân biệt, ống thẳng có 2, ô chữ thập có 1. Với bàn 4×4 điển hình $U = 4^{16} \approx 4{,}29 \times 10^9$; với 7×7 cận trên là $4^{49} \approx 3{,}17 \times 10^{29}$. Cây gán có độ sâu n, hệ số phân nhánh tối đa 4, và **mỗi tiền tố có đúng một cha** (thứ tự gán cố định), nên không cần bảng closed để chống lặp trạng thái.

Trước khi tìm kiếm, các định hướng chìa cổng ra ngoài biên bị loại. Với mỗi ứng viên con, hàm `compatible()` dùng chung cho cả hai giải thuật thực hiện ba lớp cắt tỉa hợp lệ (sound):

1. **Khớp với láng giềng đã gán**: mỗi miền R_i chưa gán bị thu hẹp chỉ giữ định hướng khớp cổng của các ô đã gán (cả cổng có lẫn cổng *không* có). Miền nào rỗng thì chặn nhánh.
2. **Chặn chu trình**: nếu các ô đã gán tạo chu trình thì loại (dùng cấu trúc disjoint-set).
3. **Kết nối khả dĩ**: dựng đồ thị *lạc quan* giữa các ô — có cạnh nếu hai ô *có thể* mở cổng về phía nhau với một định hướng còn lại nào đó. Nếu đồ thị này không liên thông thì không thể có mạng hoàn chỉnh, chặn nhánh.

Tại gốc, kiểm tra nhanh $2(n-1)$ cổng: mạng cây trên n đỉnh có đúng $n-1$ cạnh $= 2(n-1)$ cổng; xoay không đổi số cổng nên tổng sai chứng tỏ vô nghiệm ngay từ đầu. Điều ngược lại không đúng (đủ cổng chưa chắc có nghiệm). Cả ba lớp cắt tỉa đều chỉ loại trạng thái *không thể* dẫn tới nghiệm nên không mất tính đầy đủ. DFS do đó là giải thuật mù về *thứ tự duyệt*, nhưng vẫn là DFS có cắt tỉa ràng buộc chứ không phải máy đếm xoay vét cạn.

2.4 Giải thuật tìm kiếm mù: Depth-First Search

DFS đi sâu theo nhánh con khả thi đầu tiên cho đến khi gặp đích hoặc bế tắc rồi quay lui. Thứ tự duyệt các xoay là thứ tự lưu sẵn của từng ô — hoàn toàn không dùng thông tin heuristic:

```
SEARCH_DFS(input):
  build boundary-valid domains R_i
  if root feasibility fails: return UNSOLVABLE
  return VISIT(())

VISIT(s):
  if budget exhausted: return LIMIT_REACHED
  count one expansion
  if GOAL(s): return SOLVED(s)
  for a in boundary-valid rotations of tile |s|:
    child = s + (a)
    if FEASIBLE(child):           # 3 pruning layers
      r = VISIT(child)
      if r is SOLVED or LIMIT_REACHED: return r
  return UNSOLVABLE
```

Với không gian hữu hạn, cây có chu kỳ bị cắt bởi cấu trúc tiền tố, và cắt tỉa là hợp lệ nên DFS *đầy đủ* nếu đủ tài nguyên. Lời giải đầu tiên tìm được không đảm bảo tối ưu số phép gán. Độ phức tạp thời gian xấu nhất là hàm mũ theo n; về bộ nhớ, DFS giữ đường hiện hành và các nhánh chưa xét — cài đặt Python giữ bản sao tiền tố và khung generator cho mỗi mức, khoảng $O(n^2)$ trên một nhánh.

2.5 Giải thuật heuristic: Greedy Best-First

Greedy luôn pop khỏi hàng ưu tiên (min-heap) trạng thái có ít *lựa chọn còn lại* nhất:

$$h(s) = \sum_{i \text{ chưa gán}} \log_2 |D_i(s)|, \qquad \text{priority}(s) = (h(s),\ -\text{depth}(s),\ \text{insertion order})$$

với $D_i(s)$ là miền biên-hợp-lệ của ô i sau khi khớp với các láng giềng đã gán. Tổng là logarit của tích các miền còn lại: ví dụ miền $(1, 2, 4)$ cho $h = 0+1+2 = 3$, còn $(1, 1, 2)$ cho $h = 1$ — trạng thái B ít "tự do" hơn và được ưu tiên. Nhóm chọn heuristic này vì: (i) nó rẻ, chỉ cần miền đã tính trong bước cắt tỉa; (ii) mọi lời giải ở cùng độ sâu đều cần đúng n phép gán nên $n-k$ không phân biệt được gì, còn tổng log-miền ưu tiên nhánh ít phân nhánh nhất. h ở đây *không admissible* — nó đo độ bất định còn lại, không phải cận dưới chi phí; và không dùng thành phần g.

```
SEARCH_GREEDY(input):
  build boundary-valid domains; root feasibility check
  push () into min-priority queue with priority 0
  while queue is not empty:
```

```
  if budget exhausted: return LIMIT_REACHED
  s = pop minimum-priority state;  count one expansion
  if GOAL(s): return SOLVED(s)
  for a in boundary-valid rotations of tile |s|:
    child = s + (a);  D = FEASIBLE(child)
    if D exists: push child with (h(child), -|child|, serial)
return UNSOLVABLE
```

Trên không gian tiền tố hữu hạn, vô chu kỳ, việc giữ toàn bộ ứng viên khả thi làm Greedy đầy đủ nếu đủ tài nguyên; nhưng Greedy phải giữ cả frontier trong heap nên bộ nhớ có thể tăng hàm mũ.

2.6 Hiện thực và demo trực quan

Cài đặt nằm trong package `pipes_problem`: `model.py` (luật, validation, sinh bàn), `search.py` (hai giải thuật, cắt tỉa, đo thời gian/bộ nhớ), `app.py` (giao diện pygame), `worker.py` (process đo độc lập), `benchmark.py` (chạy batch). Giao diện demo:

- Chơi thủ công: click xoay (Shift để ngược chiều), right-click khóa ô, Undo/Reset; ống có nước hiển thị màu xanh chảy từ nguồn.
- **Demo tìm kiếm từng bước**: chọn DFS hoặc Greedy rồi *Run / Pause / Next / Speed*. Mỗi sự kiện hiển thị đúng loại: *expand* (thăm/pop tiền tố), *assign* (DFS chọn con), *queue* (Greedy push), *reject* (định hướng bị cắt tỉa — ô viền đỏ kèm lý do), *backtrack*. Nút *Next* cho phép phân biệt *mở rộng trạng thái* với *sinh successor*.
- *Show solution* phát lại riêng bộ gán cuối cùng; bàn trở về trạng thái đầu trước khi chạy. Kết quả *vô nghiệm* và *chạm giới hạn* hiển thị thành hai thông báo phân biệt.

2.7 Tạo input

Nhóm sinh input bằng generator của project (không chép tay từ web): 9 map mặc định gồm 3 kích thước $4 \times 4, 5 \times 5, 7 \times 7 \times 3$ seed 261–263, định dạng JSON gọn (`size`, `source`, `tiles`), tên file là định danh input. Ngoài ra có `maps/custom/cdai.json` — map được hiệu chỉnh tay thành *vô nghiệm* để kiểm tra thông báo và hành vi cạn không gian. Generator runtime cũng cho sinh map theo `--size`/`--seed`. Đây là input *mẫu của nhóm* (không phải map chính thức của trang web) và generator không lưu nghiệm vào file map.

2.8 Kết quả thực nghiệm: thời gian và bộ nhớ

Giao thức đo. Chạy trên máy Linux x86_64 (kernel 6.8), Python 3.13.14, pygame-ce 2.5.8. Cả hai giải thuật nhận *cùng* 9 map, cùng thứ tự gán row-major, cùng bộ cắt tỉa, giới hạn 30 giây và 200,000 lần mở rộng mỗi lượt. Mỗi cặp (map, giải thuật) chạy 5 lần: thời gian là solve headless bằng `perf_counter` (không vẽ hình); bộ nhớ là đỉnh cấp phát Python đo bằng `tracemalloc` trong một replay riêng, chỉ gộp khi hai lần cho cùng status, cùng bàn kết quả và cùng số lần mở rộng. Bảng số liệu đầy đủ (90 lượt) lưu tại `docs/benchmarks/2026-10-25-pipes.csv`; các bảng dưới là trung vị (median) theo từng input.

Giải thích kết quả.

Input	Giải thuật	Solved/lượt	Thời gian (ms)	Bộ nhớ (KiB)
pipes-4x4-261	DFS	5/5	3.380	26.836
	Greedy	5/5	2.866	11.695
pipes-4x4-262	DFS	5/5	3.047	27.078
	Greedy	5/5	3.386	12.359
pipes-4x4-263	DFS	5/5	3.149	26.984
	Greedy	5/5	2.741	12.500
pipes-5x5-261	DFS	5/5	5.352	53.203
	Greedy	5/5	7.094	19.500
pipes-5x5-262	DFS	5/5	5.906	53.344
	Greedy	5/5	5.882	19.219
pipes-5x5-263	DFS	5/5	4.564	53.297
	Greedy	5/5	5.712	19.219
pipes-7x7-261	DFS	5/5	23.247	145.852
	Greedy	5/5	27.049	27.594
pipes-7x7-262	DFS	5/5	21.938	145.930
	Greedy	5/5	26.669	27.070
pipes-7x7-263	DFS	5/5	40.785	145.852
	Greedy	5/5	38.246	29.094

Bảng 2: Pipes — thời gian và đỉnh bộ nhớ theo từng input (median 5 lượt, đo 25/10/2026). Cả 90/90 lượt đều *solved*. Nguồn: `docs/benchmarks/2026-10-25-pipes.csv`.

- **Thời gian tăng nhanh theo kích thước** (3–5 ms ở 4×4 lên 22–41 ms ở 7×7): số biến tăng từ 16 lên 49 và chi phí kiểm tra khả thi mỗi successor tăng theo số láng giềng đã gán, đúng dự đoán hàm mũ về không gian tìm kiếm.

- **DFS nhanh hơn Greedy ở 5×5 và 2/3 map 7×7.** Trên các input dễ, cắt tỉa đã khiến nhánh đầu tiên của DFS gần như thẳng tới nghiệm; hàng ưu tiên của Greedy lúc đó chỉ tạo thêm chi phí push/pop heap và tính h mà không đổi thứ tự duyệt đáng kể. Đây là minh chứng cho việc heuristic *không miễn phí*: chi phí cài đặt phải được bù bằng chất lượng xếp hạng.

- **Trên map khó nhất (pipes-7x7-263), Greedy nhanh hơn DFS** (38.2 so với 40.8 ms): map này buộc tìm kiếm phải lui vài nhánh thật; xếp hạng theo "ít lựa chọn còn lại" giúp nhánh gấp gáp (miền thu hẹp nhanh) được xét trước, tiết kiệm bớt phần dò thừa. Chênh lệch nhỏ (5%) nhưng xuất hiện ở đúng loại input mà heuristic có giá trị.

- **Bộ nhớ: Greedy thấp hơn DFS 4–5 lần** (12–29 KiB so với 27–146 KiB). Nghịch với suy đoán lý thuyết "frontier của Greedy có thể hàm mũ", số đo tracemalloc phản ánh *cấp phát Python thực tế của cài đặt*: DFS giữ bản sao tiền tố cho từng mức đệ quy và khung generator treo (mỗi mức một tuple copy), trong khi Greedy trên input dễ chỉ giữ vài chục ứng viên do cắt tỉa gạt gần hết. Trên input khó hơn nhiều, frontier của Greedy vẫn có nguy cơ phình hàm mũ — cần thí nghiệm bổ sung để khẳng định.

- **Cả 90 lượt đều solved** trong giới hạn; không có run nào chạm 30 giây hay 200k expansions trên bộ input này.

Kích thước	Giải thuật	Solved/lượt	Median time (ms)	Median memory (KiB)
4×4	DFS	15/15	3.149	26.984
	Greedy	15/15	3.097	12.359
5×5	DFS	15/15	5.261	53.297
	Greedy	15/15	6.157	19.219
7×7	DFS	15/15	24.469	145.852
	Greedy	15/15	31.695	27.594

Bảng 3: Pipes — tổng hợp theo kích thước (3 map $\times$ 5 lượt mỗi ô số liệu).

2.9 Hạn chế

Bộ input mặc định khá dễ với cắt tỉa hiện tại nên hai giải thuật chưa bị ép tới giới hạn; kết luận về "kẻ thắng" chỉ có giá trị trên loại input này. Nhóm chưa đo riêng đóng góp từng lớp cắt tỉa (ablation), chưa thêm đủ nhiều map vô nghiệm, và chưa thử heuristic MRV. Các hạn chế này được ghi nhận trong `docs/pipes-analysis.md` và là việc làm tiếp theo.

3 Bài toán 2: Light Up (Akari)

3.1 Tìm hiểu đề bài và luật chơi

Nhóm tìm hiểu Light Up (tên khác: Akari) qua trang https://www.puzzle-light-up.com/, cùng hệ bài toán Logic Puzzles với Pipes. Luật tóm tắt sau khi chơi thử:

- Bàn vuông gồm *ô trắng*, *tường đen không số* và *tường đen có số* (0–4). Nhiệm vụ: đặt bóng đèn vào các ô trắng.
- Mỗi bóng chiếu sáng theo 4 hướng (lên–xuống–trái–phải) cho tới khi bị tường hoặc biên chặn; ô có bóng cũng được tính là sáng.
- **Ràng buộc 1**: mọi ô trắng phải được chiếu sáng.
- **Ràng buộc 2**: hai bóng không được chiếu thẳng vào nhau (trên cùng hàng/cột, không có tường ở giữa).
- **Ràng buộc 3**: tường số k phải có đúng k bóng kề trực tiếp (4 hướng).

So với Pipes, bài toán này có thêm trục *chi phí*: trong game gốc lời giải là duy nhất (hoặc gần duy nhất), nhưng ở dạng tổng quát có thể có nhiều cách đặt đèn với số bóng khác nhau. Nhóm chọn mục tiêu tối ưu cho giải thuật heuristic: **A* trả về lời giải dùng ít bóng nhất**, trong khi DFS trả về lời giải đầu tiên tìm được. Đây là khác biệt quan trọng khi diễn giải số liệu thực nghiệm.

3.2 Phát biểu bài toán tìm kiếm

Gọi W là số ô trắng. Quyết định biểu diễn trạng thái có ảnh hưởng lớn nhất đến hiệu năng: mỗi trạng thái là một **bitmask nguyên** W bit, bit i bật nghĩa ô trắng i có bóng. Đặt bóng, kiểm tra xung đột, tính ô được chiếu đều trở thành phép toán trên số nguyên:

Cả hai giải thuật đều **rẽ nhánh theo ô trắng tối đầu tiên chưa được chiếu (theo thứ tự row-major)**: chỉ xét các bóng hợp lệ có thể chiếu ô đó. Quy tắc này làm không gian gán có cấu

Thành phần	Định nghĩa trong cài đặt
Input	Bàn: `None` (trắng), -1 (tường), $0..4$ (tường số)
Trạng thái	Tập ô có bóng, mã hóa bitmask W bit
Trạng thái đầu	Tập rỗng (mask 0)
Tác động	Đặt bóng vào một ô trắng hợp lệ
Chuyển tiếp	Bật bit của ô; loại successor vi phạm luật hoặc mất khả năng đạt
Kiểm tra đích	Mọi ô trắng sáng, mọi tường số đúng, không xung đột bóng
Chi phí	1 mỗi bóng đặt vào (trục tối ưu của A*)
Lời giải	Một tập bóng đạt đích + thứ tự đặt (path)

Bảng 4: Phát biểu bài toán tìm kiếm cho Light Up.

trúc cây (mỗi trạng thái một cha), chống lặp tự nhiên, và *giống nhau cho cả hai giải thuật* nên phép so sánh là công bằng. Trước khi nhận một successor, hàm `feasible` dùng chung kiểm tra: không xung đột tầm nhìn, không tường số nào bị vượt; mọi tường số còn *đủ* ô kề hợp lệ để đạt số cần; và không tồn tại ô tối nào *không còn bóng hợp lệ nào chiếu được*. Đây đều là cắt tỉa hợp lệ — chỉ loại trạng thái không thể dẫn tới nghiệm.

3.3 Giải thuật tìm kiếm mù: Depth-First Search

DFS dùng ngăn xếp LIFO cho các bitmask chờ xét, kèm tập visited chống quay lại trạng thái đã thấy. Thứ tự rẽ nhánh là "ô tối đầu tiên + thứ tự row-major", hoàn toàn không dùng thông tin heuristic:

```
DFS(puzzle, limits):
  push mask 0 onto the stack
  while stack is not empty:
    if budget exhausted: return TIMEOUT / NODE_LIMIT / MEMORY_LIMIT
    state = stack.pop()
    if state in visited: continue;  mark visited
    if GOAL(state): return SOLVED(state)
    cell = FIRST_UNLIT_CELL(state)        # row-major policy
    for c in legal bulbs illuminating cell:
      child = state | bit(c)
      if FEASIBLE(child): push child
  return UNSOLVABLE
```

DFS đầy đủ trên không gian hữu hạn với cắt tỉa hợp lệ, nhưng *lời giải đầu tiên không tối ưu* số bóng. Độ phức tạp xấu nhất $O(2^W)$ trạng thái; bộ nhớ giữ visited và ngăn xếp, cũng hàm mũ trong trường hợp xấu nhất. Giới hạn thời gian / số node / bộ nhớ ước tính đều trả về trạng thái kết thúc *phân biệt* với UNSOLVABLE — một lượt TIMEOUT không phải bằng chứng vô nghiệm.

3.4 Giải thuật heuristic: A* với cận dưới admissible

A* duyệt theo $f(n) = g(n) + h(n)$ với $g(n)$ = số bóng đã đặt (chi phí đơn vị), hàng ưu tiên là min-heap, kèm bảng `best_g` bỏ qua trạng thái được tái khám với chi phí không tốt hơn. Thành phần quyết định là heuristic:

$$h(n) = \max\left\{ \left\lceil \frac{D}{G_{\max}} \right\rceil,\ \max_w d_w,\ \left\lceil \frac{\sum_w d_w}{H_{\max}} \right\rceil \right\}$$

với D = số ô tối còn lại, $G_{\max}$ = số ô tối đa *một* bóng hợp lệ chiếu được ở trạng thái hiện tại, d_w = phần thiếu của tường số w (số bóng còn phải đặt kề nó), $H_{\max}$ = số kề của một tường mà *một* bóng hợp lệ có thể thỏa cùng lúc. Ba thành phần đều là **cận dưới** số bóng còn phải đặt: mỗi bóng chiếu tối đa $G_{\max}$ ô; mỗi tường thiếu d_w cần ít nhất d_w bóng; tổng phần thiếu $\sum d_w$ cần ít nhất $\lceil \sum d_w / H_{\max} \rceil$ bóng. max của các cận dưới vẫn là cận dưới, nên h **admissible**; với chi phí đơn vị và tìm kiếm chạy hết, A trả về lời giải **tối ưu số bóng**.

```
ASTAR(puzzle, limits):
  push (f = h(0), g = 0, mask 0) into a min-heap
  best_g = {0: 0}
  while heap is not empty:
    if budget exhausted: return TIMEOUT / NODE_LIMIT / MEMORY_LIMIT
    pop state with minimum f
    if GOAL(state): return SOLVED(state)
    for c in legal bulbs illuminating FIRST_UNLIT_CELL(state):
      child = state | bit(c)
      if FEASIBLE(child) and g+1 < best_g.get(child, inf):
        best_g[child] = g + 1
        push (g + 1 + h(child), g + 1, child)
  return UNSOLVABLE
```

Cái giá: heap phải giữ *toàn bộ frontier*, bảng `best_g` giữ mọi trạng thái đã thấy, và mỗi lần mở rộng phải tính lại ba cận dưới trên bitmask lớn. Đây chính là nguồn của chênh lệch thời gian và bộ nhớ thấy được trong thực nghiệm.

3.5 Hiện thực và demo trực quan

Cài đặt nằm trong package `light_up_problem`, giữ cấu trúc của project gốc: `core/` (mô hình bàn, luật, JSON), `algorithms/search.py` (hai giải thuật bitmask), `generator/` (sinh bàn theo seed), `benchmark/` (CLI đo lường), `ui/app.py` (game pygame + pygame_gui), `worker.py` (process solver spawn-safe). Giao diện demo:

- **Chơi thủ công**: click đặt/bỏ bóng; ô được chiếu tô màu sáng, viền đỏ báo xung đột, tường số hiển thị số bóng kề hiện tại.
- **Demo tìm kiếm từng bước**: chọn DFS hoặc A*, nhấn Run/Pause/Step/ Stop; các sự kiện tìm kiếm được stream về UI (đã lấy mẫu) hiển thị bộ bóng hiện tại, ô đang xét, bộ đếm expanded/generated/frontier. *Play solution* phát lại từng bước đặt bóng của lời giải.
- **So sánh DFS và A***: chạy cả hai giải thuật trên cùng bàn và hiển thị kết quả cạnh nhau (thời gian, node, số bóng).

3.6 Tạo input

Bàn chơi được sinh xác định theo seed bởi `generator/generate.py`: xây *witness* dạng hoán vị tuần hoàn (mỗi hàng/cột đúng một bóng), đề xuất tường chỉ ở nơi không phá witness, suy ra số trên tường từ các bóng kề witness, và chỉ giữ tường nếu witness vẫn là nghiệm hợp lệ. Witness được kiểm bằng chính hàm `is_goal_state` dùng khi chơi, nhưng *không* lưu vào bàn đưa cho người chơi. Có 12 bộ sample phủ đủ 4 kích thước (7, 10, 14, 25) $\times$ 3 độ khó (easy/normal/hard, mật độ tường mục tiêu 16%/10.5%/5.5% diện tích). Input JSON định dạng `lightup-puzzle-v1`

xuất/nhập bằng `save_puzzle`/ `load_puzzle` hoặc qua `--puzzle-json`. Hai giới hạn nói rõ trong báo cáo: *độ khó gán theo cấu trúc* (mật độ tường), chưa hiệu chuẩn theo thời gian giải; và *tính duy nhất nghiệm chưa được kiểm tra*.

3.7 Kết quả thực nghiệm: thời gian và bộ nhớ

Giao thức đo. Máy Linux x86_64 (kernel 6.8), Python 3.13.14 — cùng môi trường với bảng Pipes. 12 input sample (4 kích thước $\times$ 3 độ khó), mỗi giải thuật chạy 3 lượt, giới hạn 30 giây, 500,000 node, 512 MB bộ nhớ ước tính. Thời gian do solver tự đo bằng `perf_counter`; bộ nhớ là đỉnh cấp phát Python qua `tracemalloc` bao quanh lời gọi solver. Toàn bộ 72 lượt nằm ở `docs/benchmarks/2026-10-25-lightup.csv`; bảng dưới là median 3 lượt. Giới hạn 30 giây là *thiết kế thí nghiệm của nhóm* (đề bài không quy định giới hạn thời gian) để hai giải thuật dùng chung một ngân sách; riêng với 7 input mà A* vượt ngưỡng, nhóm chạy thêm A* một lượt với ngân sách 600 giây (gấp 20 lần) — Bảng 6 cho biết những lượt đó có kết thúc hay không, tránh đánh dấu TIMEOUT chỉ vì ngưỡng quá thấp.

Giải thích kết quả.

- **DFS giải được cả 12/12 input**, mở rộng rất ít node (7–35 ở hầu hết các bàn, riêng 10-easy là 248). Nguyên nhân: quy tắc "ô tối đầu tiên" buộc mỗi nhánh phủ lần lượt các ô theo thứ tự, còn cắt tỉa `feasible` loại sớm các bóng xung đột; trên các bàn nhiều tường số, hầu như không phải quay lui — node cuối chỉ gấp 1.3 số bóng. Thời gian tăng theo kích thước (0.02 s ở 7$\times$7 lên 6.7–9.7 s ở 25$\times$25) chủ yếu do chi phí thao tác bitmask và kiểm tra khả thi trên $W \approx 500$–600 ô trắng, chứ không phải do số node tăng.

- **A* chỉ hoàn thành 5/12**, nhưng *mọi* lời giải của A* đều là tối ưu số bóng. Trên các input cả hai cùng giải xong, DFS khớp tối ưu ở 2 bàn (7-easy: 7 bóng; 7-hard: 6 bóng) và thừa bóng ở 3 bàn còn lại — rõ nhất ở 10-normal: DFS dùng 12 bóng, A* chứng minh chỉ cần 9 (thừa 33%).

- **Thời gian của A* chậm hơn DFS 9–250 lần** ở nơi hoàn thành (0.27 so với 0.026 s; 26.7 so với 0.057 s). Ba nguồn chi phí: (i) heap phải giữ toàn bộ frontier thay vì một nhánh; (ii) mỗi lần mở rộng tính lại ba cận dưới bằng quét bitmask; (iii) bảng `best_g` chống tái khám phá. Số node của A* cũng cao hơn hẳn (70–5464 so với 7–13): thứ tự theo f buộc A* xử lý cả các trạng thái "rẽ ngược" mà DFS không bao giờ sinh.

- **Bộ nhớ: A* tốn hơn DFS 2–40 lần** ở nơi hoàn thành và đạt đỉnh 3621 KiB trên 10-hard trước khi TIMEOUT — frontier phình nhanh đúng như phân tích lý thuyết. Riêng nhóm 25$\times$25, bộ nhớ A* (483–833 KiB) thấp hơn DFS (553–825 KiB): A* mở rộng quá chậm (chỉ 15–37 node trong 30 giây vì mỗi lần mở rộng tốn gần giây trên bitmask 500 bit) nên chưa kịp tích frontier lớn, trong khi DFS duyệt sâu suốt 6–10 giây với visited tích lũy.

- **TIMEOUT không phải vô nghiệm — và không phải do ngưỡng 30 giây quá thấp.** Nhóm chạy riêng A* trên cả 7 input này với ngân sách 600 giây (Bảng 6): *cả 7 lượt vẫn không kết thúc*. Riêng 10-hard, A* đã mở rộng 151.546 node và giữ đỉnh cấp phát $\sim$70 MiB trong 600 giây mà frontier vẫn chưa cạn; ở nhóm 25$\times$25, chi phí mỗi lần mở rộng quá lớn ($\sim$1–2 giây trên bitmask $\sim$500 bit) nên 10 phút chỉ đạt 425–1.359 node. TIMEOUT vì thế là kết quả cạn tài nguyên *thực sự*, đúng hành vi hàm mũ của A* như lý thuyết dự đoán; không lượt nào kết thúc bằng UNSOLVABLE (mọi bàn đều có witness từ generator).

Input	Giải thuật	Kết quả	Thời gian (s)	Bộ nhớ (KiB)	Node	Bóng
akari-7-easy	DFS	SOLVED	0.0258	18.0	8	7
(7×7, seed 70097)	A*	SOLVED	0.2729	36.5	70	7
akari-7-normal	DFS	SOLVED	0.0161	20.0	9	8
(7×7, seed 70194)	A*	SOLVED	0.2355	63.2	153	**7**
akari-7-hard	DFS	SOLVED	0.0271	23.9	7	6
(7×7, seed 70291)	A*	SOLVED	6.7745	977.8	3164	6
akari-10-easy	DFS	SOLVED	0.5491	78.3	248	11
(10×10, seed 100097)	A*	SOLVED	3.4994	184.6	618	**10**
akari-10-normal	DFS	SOLVED	0.0567	47.3	13	12
(10×10, seed 100194)	A*	SOLVED	26.6845	1697.7	5464	**9**
akari-10-hard	DFS	SOLVED	0.0776	51.9	12	11
(10×10, seed 100291)	A*	TIMEOUT	30.0	3621.3	2401	—
akari-14-easy	DFS	SOLVED	0.3751	93.1	17	16
(14×14, seed 140097)	A*	TIMEOUT	30.1	750.9	347	—
akari-14-normal	DFS	SOLVED	0.4228	101.5	15	14
(14×14, seed 140194)	A*	TIMEOUT	30.1	459.8	228	—
akari-14-hard	DFS	SOLVED	0.3539	112.5	17	16
(14×14, seed 140291)	A*	TIMEOUT	30.0	1189.1	363	—
akari-25-easy	DFS	SOLVED	6.6833	553.1	33	31
(25×25, seed 250097)	A*	TIMEOUT	30.3	483.1	17	—
akari-25-normal	DFS	SOLVED	9.7177	691.5	35	34
(25×25, seed 250194)	A*	TIMEOUT	30.1	590.0	15	—
akari-25-hard	DFS	SOLVED	8.0858	824.8	33	32
(25×25, seed 250291)	A*	TIMEOUT	30.4	833.0	37	—

Bảng 5: Light Up — thời gian, bộ nhớ, số node mở rộng và số bóng theo từng input (median 3 lượt, đo 25/10/2026, ngân sách chung 30 giây). Cột "Bóng" là `solution_cost`; "—" nghĩa là lượt chạy chạm giới hạn trước khi kết luận. 72/72 lượt nằm tại `docs/benchmarks/2026-10-25-lightup.csv`.

Cần lưu ý cách đọc số liệu 26.68 giây của A* trên 10-normal: ba lượt đo đều cho 25.4–28.0 giây với *đúng* 5.464 node mở rộng (tìm kiếm tất định), nên đây là thời gian thật, không phải nhiễu đo. Chi phí mỗi node của A* trên bàn 10×10 ổn định ≈5 ms (10-easy: 3.5 s cho 618 node), nên 10-normal chậm đơn giản vì phải mở rộng gấp ≈8.8 lần số node: ít tường số hơn 10-easy khiến cận dưới yếu hơn và frontier rộng hơn — và nó nằm ngay sát mép ngưỡng 30 giây, trong khi 10-hard (càng ít tường hơn) thì vượt hẳn ngưỡng.

3.8 Hạn chế

Độ khó của bàn được gán theo mật độ tường chứ chưa hiệu chuẩn theo thời gian giải thực tế; tính duy nhất nghiệm chưa kiểm tra nên không khẳng định bàn nào có nghiệm duy nhất. Bộ nhớ đo bằng `tracemalloc` là đỉnh cấp phát Python, không phải RSS của tiến trình. Nhóm chưa thử các cận dưới mạnh hơn (ví dụ ánh xạ mỗi ô tối về ít nhất một vị trí chiếu khả dĩ rồi tính matching) — hướng có thể kéo A* đi xa hơn trước ngưỡng TIMEOUT.

Input	Thời gian (s)	Bộ nhớ (KiB)	Node mở rộng
akari-10-hard (seed 100291)	600.0	72,003	151,546
akari-14-easy (seed 140097)	600.0	16,405	19,638
akari-14-normal (seed 140194)	600.0	16,120	8,634
akari-14-hard (seed 140291)	600.0	31,244	9,477
akari-25-easy (seed 250097)	600.0	1,868	523
akari-25-normal (seed 250194)	601.8	2,649	425
akari-25-hard (seed 250291)	600.2	9,121	1,359

Bảng 6: Light Up — chạy riêng A* với ngân sách nới rộng 600 giây (1 lượt mỗi input) trên 7 input vượt ngưỡng 30 giây ở Bảng 5. **Cả 7 lượt vẫn TIMEOUT.** Dữ liệu thô: `docs/benchmarks/2026-10-25-lightup-ext.csv`.

4 Kết luận

Nhóm đã hiện thực đầy đủ hai bài toán Logic Puzzle theo yêu cầu của đề bài: mỗi bài toán có một giải thuật tìm kiếm mù (DFS) và một giải thuật heuristic (Greedy Best-First cho Pipes, A* cho Light Up), đều viết bằng Python 3, đều có giao diện demo trực quan chạy từng bước và đều được đo thời gian, bộ nhớ trên cùng bộ input với cùng giới hạn tài nguyên.

Một số bài học rút ra từ số liệu thực nghiệm:

- **Heuristic không tự động nhanh hơn mù.** Trên Pipes, Greedy Best-First chậm hơn DFS trên đa số input dễ vì chi phí duy trì heap và xếp hạng vượt lợi ích của thứ tự duyệt; chỉ trên map 7×7 khó nhất Greedy mới vượt lên. Trên Light Up, A* chậm hơn DFS rõ rệt ở 7×7 vì phải trả chi phí sắp hàng ưu tiên và quản lý trạng thái trùng — cái giá để đảm bảo lời giải tối ưu số bóng.
- **Bộ nhớ phản ánh cấu trúc frontier.** DFS giữ ít trạng thái (đường đi + nhánh đang xét) trong khi các giải thuật hàng ưu tiên phải giữ cả frontier; tuy vậy phần cài đặt Python của Pipes cho thấy DFS tốn bộ nhớ traced cao hơn do sao chép tiền tố và khung generator, còn A* của Light Up tốn cao hơn DFS do frontier và bảng trạng thái trùng.
- **Không gian trạng thái tăng hàm mũ.** Cả hai bài toán đều đạt giới hạn thời gian 30 giây ở input lớn (7×7 khó của Pipes cần gần 2 lần thời lượng trung bình; các bản Light Up 25×25 đều không giải xong trong giới hạn — kể cả khi nới ngân sách A* lên 600 giây, cả 7 input vượt ngưỡng của Light Up vẫn không kết thúc), đúng như đánh giá lý thuyết $O(4^n)$ và $O(2^W)$.
- **Cắt tỉa hợp lệ là yếu tố quyết định.** Trong cả hai bài toán, phần lớn tốc độ đến từ cắt tỉa đúng đắn (kiểm tra tính khả thi, chặn chu trình sớm, kiểm tra tính kết nối khả dĩ, chặn ô tối không thể được chiếu) chứ không phải từ thứ tự duyệt; các giải thuật mù với cắt tỉa tốt đã đủ nhanh trên input vừa và nhỏ.

Hướng phát triển: bổ sung input khó và input vô nghiệm để đánh giá rộng hơn; đo đóng góp riêng của từng lớp cắt tỉa (ablation); kiểm tra tính duy nhất nghiệm cho Light Up; thử heuristic khác (MRV cho Pipes, cận dưới mạnh hơn cho A*).

Tài liệu

[1] Puzzle Pipes. https://www.puzzle-pipes.com/

[2] Puzzle Light Up (Akari). https://www.puzzle-light-up.com/

[3] S. J. Russell, P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th edition, Pearson, 2021 (chương 3: Tìm kiếm trong không gian trạng thái; chương 6: Ràng buộc).

[4] Python 3 documentation: `time`, `tracemalloc`, `multiprocessing`, `heapq`, `unittest`. https://docs.python.org/3/

[5] pygame-ce documentation. https://pyga.me/docs/ pygame_gui documentation. https://pygame-gui.readthedocs.io/